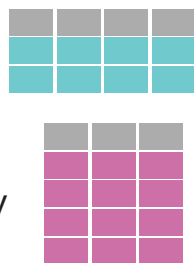


Session 3: Data wrangling

Tidy data

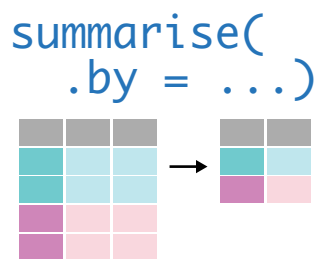
Tidy data has a regular shape: each variable is in a column, each observation is in a row and each cell only contains one value.

The process of tidying data is known as 'cleaning' or 'wrangling'



Summarising

Creates a new data frame by applying functions in a group-wise way, e.g.



```
data %>% summarise(  
  count = n(),  
  av_height = mean(height),  
  max_height = max(height),  
  .by = country)
```

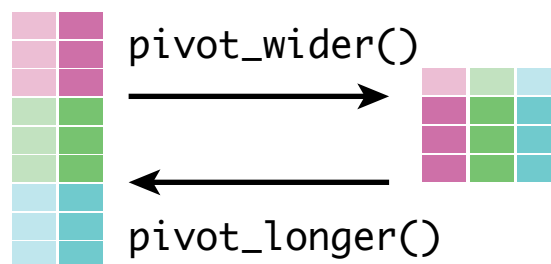
creates new data frame that contains the number of observations, average height and max height for each country. Each of these will be in its own column, with the names being on the LHS of the equal sign.

You can use summarise() without grouping to find overall summary stats.

Saving data

Use write_tsv() or write_csv() e.g.
write_tsv(data, "my_data.tsv")
saves 'data' to a .tsv file called 'my_data.tsv'

Pivoting



Pivoting allows you to change your data's shape.

pivot_wider() moves data from rows into columns, making it wider, e.g:

```
pivot_wider(  
  names_from = col_a,  
  values_from = col_b)
```

where col_a is a column that will become the new column names (pastel column above) and col_b is a column that will become the new values (bright column above)

pivot_longer() moves data from columns into rows, making it longer, e.g:

```
pivot_longer(  
  cols = c(col_x, col_y, col_z),  
  names_to = "xyz",  
  values_to = "values")
```

where cols are the columns that should become rows, names_to is the name of the column that will contain the names of the columns in cols (x, y, z) and values_to is the name of the column that will contain the values from columns x, y and z.

Summary sheet

Joining data

left_join()



Join functions allow you to combine two data frames based on a shared column. There are a variety of joins, that combine data in different ways. The left_join() function combines columns from the 'right' data frame to the 'left' data frame, for example:

```
left_join(df_left, df_right,  
  by = col_a)
```

Rows in df_right that aren't in df_left will not be included in the output.

Other useful joins include inner_join() or full_join() which keep only matching rows or all rows, respectively.

Separating columns

Sometimes, a data frame will have two variables in a single column. To make



the data tidy, we can use a function called separate():

```
separate(column_to_sep,  
  into = c("new1", "new2"),  
  sep = "_")
```

separate data in column_to_sep, using the _